Object-oriented programming

Contents

Member access 1
- Member access . . . 2
- Member access . . . 2
- Members of a struct are public by default . . . 3
- Members of a struct are public by default: Example . . . 3
- The members of a class are private by default . . . 3
- The members of a class are private by default . . . 3
- Naming your private member variables . . . 4
- Naming your private member variables . . . 4
- Naming your private member variables: Examples . . . 5
- Setting access levels via access specifiers . . . 6
- Setting access levels via access specifiers: Example . . . 6
- Access level summary . . . 6
- Access levels work on a per-class basis . . . 7
- Access levels work on a per-class basis: Example . . . 7

A detour back to pointers & class type (`struct` and `class`) 7
- What is an aggregate? . . . 8
- Aggregate initialization of a struct: Example . . . 8

Member selection with references and pointers 9
- Member selection for references . . . 9
- Member selection for pointers . . . 9
- Member selection for pointers . . . 10
- Member selection for pointers . . . 10
- What's next . . . 10

Introduction to constructors 11
- Intro to constructors: Rationale . . . 11
- Intro to constructors: Rationale . . . 12
- Constructors . . . 12
- Naming constructors . . . 13
- A basic constructor example . . . 13

Constructor member initializer lists 13
- Mimic Python constructor: Part 1 . . . 13
- Mimic Python constructor: Part 2 . . . 13
- Member initialization via a member initialization list . . . 14
- Member initialization via a member initialization list . . . 14
- Member initialization list: why better than `this->x = x` . . . 15
- `: x { x}` versus `this->x = x`: Key differences . . . 16
- Member initializer list formatting – Note C++ does not care space . . . 16

Member initializer list vs default member initializers . . . 17
Member initializer list vs default member initializers . . . 18
Constructor function bodies . . . 18

Default constructors and default arguments **18**
Default constructor . . . 18
Value initialization vs default initialization for class types . . . 18
Contructors with default argument . . . 19
Overloaded constructors . . . 19
An implicit default constructor . . . 19
An implicit default constructor . . . 19
Killing implicit default constructor . . . 20
Using `= default` to generate an explicitly defaulted default constructor . . . 20
Using `= default` to generate an explicitly defaulted default constructor . . . 20

Intro to the copy constructor **20**
Intro to the copy constructor . . . 20
Intro to the copy instructor . . . 21
Copy constructor . . . 21
An implicit copy constructor . . . 21
Copy constructor . . . 21
Copy constructor: Best practices . . . 21
The copy constructor's parameter must be a reference . . . 21
Pass by value and the copy constructor . . . 21
Return by value and the copy constructor . . . 21
Return by value and the copy constructor . . . 22
Using `= default` to generate a default copy constructor . . . 22
Using `= delete` to generate a default copy constructor . . . 22

Member access

We have used structs in all the example in the last Chapter.

- This demonstrated that `struct` is pretty much the same as `class`.
- The difference between `struct` and `class` is
  - By default, every data members and member functions of a `struct` is `public` by default.
  - By default, every data members and member functions of a `struct` is `private` by default.

We study now **access level** to debunk the meaning of two keywords `private` and `public`.

Member access

You feel tired on the way going home and want to sit down in peace as you are an introvert. Two choices:

- The park with all the screaming kids. Not want to hear all the screamings. It is `public`, you can access.
- The private residential house with warm and TV. Want to sit in silence. It is `private`, you cannot access.

It is not about what you want, but about what you can.

Member access

Each member of a class type (`struct` or `class`) have a property called an **access level**.

Access level determines who can access that member:

1. `public`
2. `private`
3. `protected`

We'll cover

- `public` and `private` in this Chapter
- `protected` in Chapter "Inheritance".

Members of a struct are public by default

Terminologies

- Members that have the public access level are called **public members**.
- **Public members** are members of a class type that do not have any restrictions on how they can be accessed.

Meaning

- Public members can be accessed by other members of the same class.
- **Public members can also be accessed by the public**, which is what we call code that exists outside the members of a given class type.

Key points to remember

- The members of a struct are public by default.
- Public members can be accessed by other members of the class type, and by public

Members of a struct are public by default: Example

We have done this all the time before:

The members of a class are private by default

Now, look at this example again but using `class` instead of `struct`:

The members of a class are private by default

Key points to remember

- The members of a class are private by default.
- Private members can be accessed by other members of the class type, **but cannot be accessed by public**

Repeat

`struct` and `class` are only different by technical detail, but essentially interchangeable:

- To make `struct` look like `class`, we make all the members `private`.
- To make `class` look like `struct`, we make all the members `public`.

Listing 1 All the members of a struct are `public` by default. Filename=`struct_public_by_default.cpp`

```cpp
#include <iostream>

struct Date
{
    // struct members are public by default, can be accessed by anyone
    int year {};       // public by default
    int month {};      // public by default
    int day {};        // public by default

    void print() const // public by default
    {
        // public members can be accessed in member functions of the class type
        std::cout << year << '/' << month << '/' << day;
    }
};

// non-member function main is part of "the public"
int main()  // main is not a member of Date, it is part of `public`.
{
    Date today { 2020, 10, 14 }; // aggregate initialize our struct

    // public members can be accessed by the public
    today.day = 42;     // okay: the day member is public. That's why it's dangerous
    today.month = 42;   // okay: the month member is public. That's why it's dangerous
    today.year = -42;   // okay: the year member is public. That's why it's dangerous
    today.print();      // okay: the print() member function is public

    return 0;
}
```

Naming your private member variables

It is a common convention to *name private data members starting with an* `m_` *prefix.* But ... talk later.

Consider:

```cpp
// Some member function that sets private member m_name to
// the value of the name parameter
void set_name(std::string_view name)
{
    m_name = name;
}
```

- The `m_` prefix differentiates data members from function parameters or local variables.
- The `m_` prefix helps prevent naming collisions between private member variables and the names of local variables.

Naming your private member variables

If we had named private members `name` instead of `m_name`, it looks like this

Listing 2 All the members of a class are `private` by default. Filename=`class_private_by_default.cpp`

```cpp
#include <iostream>

class Date // now a class instead of a struct
{
    // class members are private by default, can only be accessed by other members
    int year {};     // private by default
    int month {};    // private by default
    int day {};      // private by default

    void print() const // private by default
    {
        // private members can be accessed in member functions
        std::cout << year << '/' << month << '/' << day;
    }
};

int main()  // main is a part of public
{
    // Date today { 2020, 10, 14 }; // compile error: can no longer use aggregate initialization

    // private members can not be accessed by the public
    // You cannot do something silly like this.
    /*
    today.m_day = 42;   // compile error: the m_day member is private.
    today.m_month = 42; // compile error: the m_month member is private.
    today.m_year = 42;  // compile error: the m_year member is private.
    today.print();      // compile error: the print() member function is private.
    */
    return 0;
}
```

```cpp
// then we need to change the parameter's name here
void set_name(std::string_view name)
{
    // function parameter "name" shadows the private data member "name"
    name = name // of course won't work
}
```

But ... soon we will see this is even not necessary in modern C++.

Well, why do I need to mention?

- You need to read legacy code when you go to industry.
- But the use of prefix `m_` is declining.
- Many high-profile open-source projects have moved away from `m_` in **favor of a trailing underscore** (`variable_`) or **no prefix at all**

Naming your private member variables: Examples

Examples – Click on the links to show the students examples

- **Google**: trailing underscore `variable_`
- **PyTorch** (famous Machine Learning framework): no prefix, no suffix

  Link to one example, Lines 68 – 75
- **deal.II** (famous finite element library): no prefix, no suffix

  Link to one example, Lines 2522 – 2721

Morale of the story

- All the examples in real-world libraries/application look extremely complicated.
- But ... *Hey! You want to earn 6-digit salary and want it to look simple?*

I am not against GenAI chatbot. You can hate me now but I want to say this:

You want to get the job in industry after using ChatGPT/Gemini to generate solution for the programming courses?

Just think about that!?!?

Setting access levels via access specifiers

We can explicitly set the access level of our members by using an access specifier.

An access specifier sets the access level of all members that follow the specifier.

Three access specifiers:

- `public:`
- `private:`
- `protected:`

Setting access levels via access specifiers: Example

We can omit `private:` specifier in `class` as classes default to `private` access.

Let us compile and run this code.

Access level summary

Access level	Access specifier	Member access	Derived class access	Public access
Public	`public:`	yes	yes	yes
Protected	`protected:`	yes	yes	no
Private	`private:`	yes	no	no

Don't worry! We talk about this later.

- Some little practice will get you understand and remember them.
- However, notice the triangle pattern of "yes" and triangle pattern of "no".

Listing 3 We can use access specifiers to define access levels of members. Filename=`access_specifier_example.cpp`

```cpp
#include <iostream>
class Date
{
// Any members defined here would default to private

public: // here's our public access specifier

    void print() const // public due to above public: specifier
    {
        // members can access other private members
        std::cout << year << '/' << month << '/' << day;
    }

private: // here's our private access specifier

    int year { 2020 };  // private due to above private: specifier
    int month { 14 }; // private due to above private: specifier
    int day { 10 };   // private due to above private: specifier
};

int main()
{
    Date d{};
    d.print();  // okay, main() allowed to access public members

    return 0;
}
```

Access levels work on a per-class basis

One nuance of C++ access levels that is often missed or misunderstood is that – otherwise, it is a real big headache:

Access to members is defined on a per-class basis, not on a per-object basis.

Access levels work on a per-class basis: Example

We can access member `name` of the object `p` in the member function `kisses`.

`p.name` is not private to only `p`. Any member functions of `Person` can access `name` of other `Person` objects.

A detour back to pointers & class type (`struct` and `class`)

You have just seen `this->name`.

- But we skip the lecture on `struct` on purpose.
- I explained why we skip the lecture on `struct`.
- Now, I have to pay the price. We must make a detour to revisit pointers.

Listing 4 Access levels work on per-class basis, not on object basis. Filename=`access_level_per_class_basis.cpp`

```cpp
#include <iostream>
#include <string>
#include <string_view>

class Person
{
private:
    std::string name{};

public:
    void kisses(const Person& p) const
    {   // we access data member "name" of object "p"
        std::cout << name << " kisses " << p.name << '\n';
    }

    void set_name(std::string_view name)
    {   // this is the pointer to the "current" object.
        // This is why we don't need to use m_name, or name_
        // and everything still appears normal and sensible.
        this->name = name;
    }
};

int main()
{
    Person joe;
    joe.set_name("Joe");
    Person kate;
    kate.set_name("Kate");
    joe.kisses(kate);

    return 0;
}
```

What is an aggregate?

- In general programming, an aggregate data type (also called an aggregate) is any type that can contain multiple data members.
- Some types of aggregates allow members to have different types (e.g. structs).
- Others require that all members must be of a single type (e.g. arrays).

In our course:

When we use the term "aggregate" (or "non-aggregate") we will mean the C++ definition of aggregate.

Aggregate initialization of a struct: Example

Best practice: Prefer the (non-copy) braced list form when initializing aggregates.

Listing 5 We can initialize aggregate by using copy-list initialization or list initialization. Filename=`aggregate_initialization.cpp`

```cpp
#include <iostream>
struct Employee
{
    int id {};          // default initialization
    int age {42};       // default initialization
    double wage {};     // default initialization
};

int main()
{
    // copy-list initialization using braced list
    Employee frank = { 1, 32, 60000.0 };
    // list initialization using braced list (preferred)
    Employee joe { 2, 28, 45000.0 };
    // id, age and wage will be initialized by default as above
    Employee khiem {};
    std::cout << "khiem.age = " << khiem.age << "\nkhiem.wage = " << khiem.wage << '\n';
    return 0;
}
```

Member selection with references and pointers

Member selection for references

Because references to an object act just like the object itself, we can also use the member selection operator (`.`) to select a member from a reference to a struct

Member selection for pointers

We know that this code is wrong – **Question**: What is wrong here?

```cpp
#include <iostream>

struct Employee
{
    int id{};
    int age{};
    long double wage{};
};

int main()
{
    Employee khiem{ 1, 42, 429999999999999999999.42 };
    Employee* ptr { &khiem };
    std::cout << ptr.id << '\n';
    return 0;
}
```

My salary is correct. It is not the error!

Listing 6 Reference to a struct object can selects members by using selection operator (.) Filename=`reference_selection_operator.cpp`

```cpp
#include <iostream>

struct Employee
{
    int id{};
    int age{};
    double wage{};
};

void print_employee(const Employee& employee)
{
    // Use member selection operator (.) to select member from reference to struct
    std::cout << "Id: " << employee.id << '\n';
    std::cout << "Age: " << employee.age << '\n';
    std::cout << "Wage: " << employee.wage << '\n';
}

int main()
{
    Employee joe{ 1, 34, 65000.0 };
    joe.wage = 68000.0;
    print_employee(joe);

    return 0;
}
```

Member selection for pointers

To fix this, we just need to dereference first to retrieve the struct object.

Now, both my salary and code are fine.

Member selection for pointers

C++ offers a **member selection from pointer operator** (`->`) (also sometimes called the **arrow operator**) that can be used to select members from a pointer to an object.

`ptr->id` is just equivalent to `(*ptr).id`. That's all.

What's next

From now on:

- I prefer "no prefix, no suffix" naming convention, and use `this` pointer instead.
- We talk about this soon, but you kinda get the point here.

Listing 7 Pointer to a struct object can selects members by using dereference first. Filename=`pointers_to_structs_select_members_deref.cpp`

```cpp
#include <iostream>

struct Employee
{
    int id{};
    int age{};
    long double wage{};
};

int main()
{
    Employee khiem{ 1, 42, 429999999999999999999.42 };
    Employee* ptr{ &khiem };
    // Not great but works: First dereference ptr, then use member selection
    std::cout << (*ptr).id << '\n';

    return 0;
}
```

Introduction to constructors

In Introductory Programming 2, you learn:

- Constructor is defined by the special method: `__init__(self, ...)`
- Constructor is used to do a lot of stuffs other than just assigning variable `self.age = age`, e.g., checking
  - whether `age < 0`
  - whether `id` is unique
  - whether `birthdate` is a proper date
  - more

Note: We can write constructors for both `struct` and `class`. We just don't see people do that for `struct`.

We now learn constructors in C++.

Intro to constructors: Rationale

We have just learned we can do this:

```cpp
struct Foo // Foo is an aggregate
{
    int x {};
    int y {};
};

int main()
{
    Foo foo { 6, 7 }; // uses aggregate initialization
    return 0;
}
```

Listing 8 Pointer to a struct object can selects members by using dereference first. Filename=`pointers_to_structs_select_members_deref.cpp`

```cpp
#include <iostream>

struct Employee
{
    int id{};
    int age{};
    long double wage{};
};

int main()
{
    Employee khiem{ 1, 42, 429999999999999999999.42 };
    Employee* ptr{ &khiem };
    // Not great but works: First dereference ptr, then use member selection
    std::cout << (*ptr).id << '\n';

    return 0;
}
```

Intro to constructors: Rationale

But we cannot do this:

```cpp
class Foo // Foo is not an aggregate (has private members)
{
    int x {};
    int y {};
};
int main()
{
    Foo foo { 6, 7 }; // compile error: can not use aggregate initialization
    return 0;
}
```

Not allowing class types with private members to be initialized via aggregate initialization makes sense:

- Aggregate initialization requires knowing about the implementation of the class, which we're intentionally trying to avoid when we hide our data members.
- If our class had some kind of invariant (e.g., $18 < \text{age} < 65$ for working age), we would be relying on the user to initialize the class in a way that preserves the invariant.

That's why we need a constructor and it can do all the security check if needed. – just like you learned in Python.

Constructors

A **constructor** is a special member function that is automatically called after a non-aggregate class type object is created.

When a non-aggregate class type object is defined, the compiler looks to see if it can find an accessible constructor that is a match for the initialization values provided by the callers.[1]

Key insight

- Many new programmers are confused about whether constructors create the objects or not.
- They do not – the compiler sets up the memory allocation for the object prior to the constructor call. The constructor is then called on the uninitialized object.
- After the constructor finishes executing, we say the object has been "constructed", and the object should now be in a consistent, usable state.

Note

Aggregates are not allowed to have constructors – so if we add a constructor to an aggregate, it is no longer an aggregate.

Naming constructors

- Constructors must have the same name as the class (with the same capitalization).
- For template classes, this name excludes the template parameters.[2]
- Constructors have no return type (not even `void`)

```
class Student
{
private:
    std::string name;
    int age;
    int ID;

public:
    Student(std::string name, int age, int ID)
    {
        // code goes here.
    }   // Does this remind you of Python class?
}
```

A basic constructor example

Let run this code snippet.

Constructor member initializer lists

Can we do something like `self.x = x`, `self.y = y` in Python?

Mimic Python constructor: Part 1

Question: How about we want to keep the constructor's parameter names `x` and `y`?

Mimic Python constructor: Part 2

Yes, we can use the pointer `this` to make distinction: `this->x`

[1] Well, just like the compiler looks for functions and overloaded functions matching the funcation call.
[2] We have not learned about this so far.

Listing 9 Constructor must have the same name as class. Filename=`basic_constructor_example.cpp`

```cpp
#include <iostream>
class Vector2D
{
private:
    double x { 24 }; // let remove {} to illustrate non-initialized members for students
    double y {};

public:
    Vector2D(double x, double y)
    {
        this->x = x;
        this->y = y;
        std::cout << "Vector2D(" << this->x << ", " << this->y << ") constructed\n";
    }
    void print() const {
        std::cout << "Vector2D(" << x << ", " << y << ")\n";
    }
};

int main()
{
    Vector2D v {42, 24};    // call Vector2D(double, double)
    v.print();
    return 0;
}
```

Member initialization via a member initialization list

Modern C++ provides a mechanism even better than writing `this->variable = variable` in the constructor body: **member initializer list**.

Member initialization via a member initialization list

Member initializer list

- is defined after the constructor parameters.
- begins with a colon `:`
- lists each member to initialize along with the initialization value for that variable, separated by comma
- uses a direct form of initialization – *preferably braces, but parentheses work as well*

Example (again)

```cpp
class Vector2D
{
    double x {},
    double y {};
public:
    Vector2D(double x, double y): x { x }, y { y } {}
};
```

Listing 10 Let us mimic Python constructor we have learned. Filename=`mimic_python_constructor_01.cpp`

```cpp
#include <iostream>

class Foo
{
private:
    int x {};
    int y {};

public:
    // if x_out is named as x, we have shadowing
    Foo(int x_out, int y_out)
    {
        x = x_out; // we don't have self.x
        y = y_out;
        std::cout << "Foo(" << x << ", " << y << ") constructed\n";
    }

    void print() const
    {
        std::cout << "Foo(" << x << ", " << y << ")\n";
    }
};

int main()
{
    Foo foo{ 6, 7 };
    foo.print();

    return 0;
}
```

Member initialization list: why better than `this->x = x`

Member Initalizer List: `x { x }`

Why it's better:

- **initialization, not assignment**: created and given value in one single step
- **required for `const` and references**: if class has `const` members or reference variables (e.g. `double& ref`), we must use this method. These teypes cannot be left unintialized and assigned to later.
- **performance**: avoid calling default constructor followed by an assignment operator, goes straight to the copy/move constructor (discussed later)

Assignment in the Body: `this->x = x`

Sequence of events (you see it is worse than above)

- **implicit initialization**: initializes `x` with garbage value (if defined `double x`), or to default value (if defined `double x { }`)
- **execution**: constructor body starts
- **assignment**: value of the parameter `x` is copied into `this->`.

Listing 11 Let us mimic Python constructor we have learned. Filename=`mimic_python_constructor_01.cpp`

```cpp
#include <iostream>

class Foo
{
private:
    int x {};
    int y {};

public:
    Foo(int x, int y)
    {
        this->x = x; // "this" is the pointer to the object under consideration.
        this->y = y;
        std::cout << "Foo(" << x << ", " << y << ") constructed\n";
    }

    void print() const
    {
        std::cout << "Foo(" << x << ", " << y << ")\n";
    }
};

int main()
{
    Foo foo{ 6, 7 };
    foo.print();

    return 0;
}
```

`: x { x}` versus `this->x = x`: Key differences

Feature	Member Initialization List `: x { x }`	Body Assignment `this->x = x`
Action	direct initialization	Default init + assignment
Efficiency	Faster (no double-work)	Slower for complex objects
Const/Refs	Works perfectly	Fails to compile
Ordering	Follows declaration order in class	Follows code order in body

Best practice

Prefer using the member initializer list to initialize your members over assigning values in the body of the constructor.

Member initializer list formatting – Note C++ does not care space

- One line

```cpp
Foo(int x, int y) : x { x }, y { y }
{
}
```

Listing 12 An example of member initializer list. Filename=`member_initializer_list.cpp`

```cpp
#include <iostream>

class Foo
{
private:
    int x {};
    int y {};

public:
    Foo(int x, int y)
        : x { x }, y { y } // here's our member initialization list
    {
        std::cout << "Foo(" << x << ", " << y << ") constructed\n";
    }

    void print() const
    {
        std::cout << "Foo(" << x << ", " << y << ")\n";
    }
};

int main()
{
    Foo foo{ 6, 7 };
    foo.print();

    return 0;
}
```

- Multiple lines with colon `:` after class name

```cpp
Foo(int x, int y) :
    x { x },
    y { y }
{
}
```

- Multiple lines with colon `:` before the first initializer

```cpp
Foo(int x, int y)
    : x { x }
    , y { y }
{
}
```

If member initialization list is trivial, all initializers can go one line. Otherwise, third style is preferable

Member initializer list vs default member initializers

Let us consider the following examples

Member initializer list vs default member initializers

Members are initialized in a few different ways:

- If a member is listed in the member initializer list, that initialization value is used.
- Otherwise, if the member has a default member initializer, that initialization value is used.
- Otherwise, the member is default-initialized.

Constructor function bodies

- The bodies of constructors functions are most often left empty – because we primarily use constructor for initialization, which is done via member initializer list (if that's all we need to do).
- We can add statements to constructor function bodies to do any other setup tasks required (e.g. open a file or databa, allocate memory, check invariant ...)

Consider the following example

```
class Fraction
{
private:
    int numerator {};
    int denominator {};

public:
    Fraction(int numerator, int denominator):
        numerator { numerator }, denominator { denominator } {
        // Invariant: Fraction should have non-zero denominator
        // How about denominator is zero?
        // We can do something about this invariant in the ocnstructor body.
    }
};
```

Default constructors and default arguments

Default constructor

[**Default constructor**] is a constructor that accepts no arguments.

Value initialization vs default initialization for class types

If a class type has a default constructor, both value initialization and default initialization will call the default constructor.

```
// With default constructor, value initialization and default initialization will
// call the default constructor.
Foo foo1{}; // value initialization, calls Foo() default constructor
Foo foo2;   // default initialization, calls Foo() default constructor
```

Contructors with default argument

As with all functions, the rightmost parameters of constructors can have default arguments.

If all the params in a constructor have default args, the constructor is a default constructor – as it can be called with no args.

Overloaded constructors

Because constructors are functions, they can be overloaded.

```cpp
#include <iostream>

class Foo
{
private:
    int x {};
    int y {};

public:
    Foo() // default constructor
    {
        std::cout << "Foo constructed\n";
    }

    Foo(int x=1, int y=2) // default constructor
        : x { x }, y { y }
    {
        std::cout << "Foo(" << x << ", " << y << ") constructed\n";
    }
};

int main()
{
    Foo foo{}; // compile error: ambiguous constructor function call
    return 0;
}
```

The compiler will look for a default constructor & and will find two, leading to ambiguity.

An implicit default constructor

- If a non-aggregate class type (e.g. private data members) object has no user-declared constructors, the compiler will generate a public default constructor.
- This constructor is called an **implicit default constructor**

An implicit default constructor

The implicit default constructor is equivalent to a constructor that has

- no parameters
- no member initializer list
- no statements in the constructor body

```
public:
    Foo() // implicitly generated default constructor
    {
    }
```

Killing implicit default constructor

If we define any other constructor which is not a default constructor, the compiler automatically deletes the implicit default constructor.

Consequently, we need to write a default constructor if we need one. Instead of writing implicit default constructor "explicitly", we can use `= default`.

Using `= default` to generate an explicitly defaulted default constructor

Look at this example

Using `= default` to generate an explicitly defaulted default constructor

In cases where we would write a *default constructor that is equivalent to the implicitly generated default constructor*:

- We can tell the compiler to generate a default constructor for us by using the `= default` syntax.
- This constructor is called an **explicitly defaulted default constructor**.

Intro to the copy constructor

We can write code like this (try this in your assignment)

```
// Original vector
std::vector<double> original = {1.1, 2.2, 3.3, 4.4};
// List initialization (calling the copy constructor) - modern way
std::vector<double> copy1{original};
// Direct initialization (Calling the copy constructor) - old school
std::vector<double> copy2(original);
// Using the assignment syntax (Also calls copy constructor here)
std::vector<double> copy3 = original;
```

This section explains what happens behind the scene.

Intro to the copy constructor

Let us look at this code snippet. How can `Fraction f_copy { f }` run?

The answer is: **copy constructor**

Intro to the copy instructor

Copy constructor

A **copy constructor** is a constructor that is used to initialize an object with an existing object of the same type.

After the copy constructor executes, the newly created object should be a copy of the object passed in as the initializer.

An implicit copy constructor

If we do not provide a copy constructor for our classes, C++ will create a public implicit copy constructor for you.

Copy constructor

We can explicitly define our own copy constructor.

Copy constructor: Best practices

- A copy constructor should not do anything other than copy an object.
- This is because the compiler may optimize the copy constructor out in certain cases.

Best practice

Copy constructors should have no side effects beyond copying.

Best practice

Prefer the implicit copy constructor, unless you have a specific reason to create your own.

The copy constructor's parameter must be a reference

- It is a requirement that the parameter of a copy constructor must be an lvalue reference or const lvalue reference.
- The copy constructor should not be modifying the parameter, using a const lvalue reference is preferred.

Best practice

If you write your own copy constructor, the parameter should be a const lvalue reference.

Pass by value and the copy constructor

When an object is passed by value, the argument is copied into the parameter.

Let us run this program.

Return by value and the copy constructor

Return by value creates a temporary object (holding a copy of the return value) that is passed back to the caller.

The temporary object is initialized by implicitly invoking the copy constructor.

Return by value and the copy constructor

The compiler has optimized the process of making copy and eliminates unnecessary copying of objects. There should be 4 copy constructors invoked.

This is called **copy elision**.

Using `= default` to generate a default copy constructor

If a class has no copy constructor, the compiler will implicitly generate one for us.

We can explicitly request the compiler create a default copy constructor for us using the `= default` syntax:

```
// Explicitly request default copy constructor
    Fraction(const Fraction& fraction) = default;
```

Write your own example to understand this.

I provided the code already: `default_copy_constructor_using_default.cpp`; you see on GitHub & Moodle.

Using `= delete` to generate a default copy constructor

We can prevent copy by marking the copy constructor function as deleted, using the `= delete` syntax:

```
// Delete the copy constructor so no copies can be made
Fraction(const Fraction& fraction) = delete;
```

Write your own example to understand this.

I provided the code already: `remove_copy_constructor_using_delete.cpp`; you see on GitHub & Moodle.

Listing 13 Different initializers in classes. Filename=`different_initializers.cpp`

```cpp
#include <iostream>

// This function "pollutes" the stack memory with non-zero values
void do_dirty_stack() {
    int a[100];
    for(int i = 0; i < 100; ++i) {
        a[i] = 0xDEADBEEF; // A recognizable "garbage" hex value
    }
}

class Foo
{
private:
    int x {};       // default member initializer (will be ignored)
    int y { 42 };    // default member initializer (will be used)
    int z;      // no initializer, you see "0" in the output due to Compiler,
                // not real initialzation value. It may contain garbage value
                // on other computers/compilers.

public:
    Foo(int x)
        : x { x } // member initializer list
    {
        std::cout << "Foo constructed\n";
    }

    void print() const
    {
        std::cout << "Foo(" << x << ", " << y << ", " << z << ")\n";
    }
};

int main()
{
    do_dirty_stack();       // fill the stack with garbage first
    Foo foo { 6 };
    foo.print();

    return 0;
} // Compile with warning: g++ -Wall -Wextra -Wuninitialized different_initializers.cpp
```

Listing 14 A very simple default constructor example. Filename=`default_constructor_example.cpp`

```cpp
#include <iostream>

class Foo
{
public:
    Foo() { // default constructor
        std::cout << "Foo default constructed\n";
    }
};

int main()
{
    Foo foo{}; // No initialization values, calls Foo's default constructor

    // With default constructor, value initialization and default initialization will
    // call the default constructor.
    Foo foo1{}; // value initialization, calls Foo() default constructor
    Foo foo2;   // default initialization, calls Foo() default constructor
    return 0;
}
```

Listing 15 An example of constructor with default arguments. Filename=`constructor_with_default_arguments.cpp`

```cpp
#include <iostream>

class Foo
{
private:
    int x { };
    int y { };

public:
    Foo(int x=0, int y=0) // has default arguments
        : x { x }
        , y { y }
    {
        std::cout << "Foo(" << x << ", " << y << ") constructed\n";
    }
};

int main()
{
    Foo foo1{};     // calls Foo(int, int) constructor using default arguments
    Foo foo2{6, 7}; // calls Foo(int, int) constructor

    return 0;
}
```

Listing 16 Implicit default constructor is used when no default constructor is defined. Filename=`implicit_default_constructor.cpp`

```cpp
#include <iostream>

class Foo {
private:
    int x{};
    int y{};
    // Note: no constructors declared
};

int main()
{
    Foo foo{};  // implicit default constructor is used
    return 0;
}
```

Listing 17 User-defined constructor which is not default constructor would kill the implicit default constructor. Filename=`kiling_implicit_default_constructor.cpp`

```cpp
class Foo {

private:
    int x {};
    int y {};
public:
    Foo(int x, int y) : x{x}, y {y} {}  // "kills" the implicit default constructor
};

int main()
{
    Foo foo1 { 1, 2 };  // Fine. the compiler uses Foo(int, int).
    Foo foo2 {};    // error: there is no default constructor
}
```

Listing 18 Using `= default` to write a default constructor explicitly. Filename=`explicit_default_constructor.cpp`

```cpp
#include <iostream>

class Foo
{
    int x {};
    int y {};

public:
    // If we don't have this default constructor, Foo foo{} leads to error.
    // Remove this line of code to see the error.
    Foo() = default; // generates an explicitly defaulted default constructor

    Foo(int x, int y) : x { x }, y { y } {
        std::cout << "Foo(" << x << ", " << y << ") constructed\n";
    }
};

int main()
{
    Foo foo{}; // calls Foo() default constructor

    return 0;
}
```

Listing 19 An intro to copy constructor: Look at the statement `Fraction f_copy { f }`. Filename=`copy_constructor_intro.cpp`

```cpp
#include <iostream>

class Fraction
{
private:
    int numerator{ 0 };
    int denominator{ 1 };

public:
    // Default constructor
    Fraction(int numerator=0, int denominator=1)
        : numerator{numerator}, denominator{denominator}
    {
    }

    void print() const
    {
        std::cout << "Fraction(" << numerator << ", " << denominator << ")\n";
    }
};

int main()
{
    Fraction f { 5, 3 };    // Calls Fraction(int, int) constructor
    Fraction f_copy { f };  // What constructor is used here?
    f.print();
    f_copy.print();

    return 0;
}
```

Listing 20 We can explicitly define our own copy constructor. Note the pass by reference. Filename=`copy_constructor_fraction.cpp`

```cpp
#include <iostream>

class Fraction
{
private:
    int numerator{ 0 };
    int denominator{ 1 };

public:
    // Default constructor
    Fraction(int numerator=0, int denominator=1)
        : numerator{numerator}, denominator{denominator}
    {
    }

    // Copy constructor
    Fraction(const Fraction& fraction)
        // Initialize our members using the corresponding member of the parameter
        : numerator{ fraction.numerator }
        , denominator{ fraction.denominator }
    {
        std::cout << "Copy constructor called\n"; // just to prove it works
    }

    void print() const
    {
        std::cout << "Fraction(" << numerator << ", " << denominator << ")\n";
    }
};

int main()
{
    Fraction f { 5, 3 };  // Calls Fraction(int, int) constructor
    Fraction f_copy { f }; // Calls Fraction(const Fraction&) copy constructor

    f.print();
    f_copy.print();

    return 0;
}
```

Listing 21 When an object is passed by value, the argument is copied into the parameter. Filename=`copy_constructor_pass_by_value.cpp`

```cpp
#include <iostream>

class Fraction
{
private:
    int numerator{ 0 };
    int denominator{ 1 };

public:
    // Default constructor
    Fraction(int numerator = 0, int denominator = 1)
        : numerator{ numerator }, denominator{ denominator }
    {
    }

    // Copy constructor
    Fraction(const Fraction& fraction)
        : numerator{ fraction.numerator }
        , denominator{ fraction.denominator }
    {
        std::cout << "Copy constructor called\n";
    }

    void print() const
    {
        std::cout << "Fraction(" << numerator << ", " << denominator << ")\n";
    }
};

void print_fraction(Fraction f) // f is pass by value
{
    f.print();
}

int main()
{
    Fraction f{ 5, 3 };

    print_fraction(f); // f is copied into the function parameter using copy constructor

    return 0;
}
```

Listing 22 When a function returns by value, a temporary object is initialized by implicitly invoking the copy constructor and is returned to the caller. Filename=`copy_constructor_return_by_value.cpp`

```cpp
#include <iostream>

class Fraction
{
private:
    int numerator{ 0 };
    int denominator{ 1 };

public:
    // Default constructor
    Fraction(int numerator = 0, int denominator = 1)
        : numerator{ numerator }, denominator{ denominator }
    {
    }

    // Copy constructor
    Fraction(const Fraction& fraction)
        : numerator{ fraction.numerator }
        , denominator{ fraction.denominator }
    {
        std::cout << "Copy constructor called\n";
    }

    void print() const
    {
        std::cout << "Fraction(" << numerator << ", " << denominator << ")\n";
    }
};

void print_fraction(Fraction f) // f is pass by value
{
    f.print();
}

Fraction generate_fraction(int n, int d)
{
    Fraction f{ n, d };
    return f;       // one copy constructor invoked
}

int main()
{
    Fraction f1 {};
    print_fraction(f1); // one copy constructor invoked
    Fraction f2 { generate_fraction(1, 2) }; // Fraction is returned using copy constructor

    print_fraction(f2); // f is copied into the function parameter using copy constructor
    print_fraction(f2);

    Fraction f3 { generate_fraction(4, 2) };
    print_fraction(f3);
    return 0;
}
```